Chuleta de Git para hardware

Configuración (una vez por computadora)

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu-correo@pucp.edu.pe"
git config --global init.defaultBranch main
git config --global core.editor "code --wait"
```

¿Dónde estoy?

```
pwd                             # carpeta actual
ls                              # qué hay aquí
cd demogit                      # entrar a una carpeta
cd ..                           # subir un nivel
explorer .                      # Windows: abrir esta carpeta en el Explorador
open .                          # macOS: abrir esta carpeta en Finder
```

El ciclo diario

```
git status                        # ¿qué cambió? (úsalo TODO el tiempo)
git diff                          # ver los cambios línea por línea
git add DemoGit.kicad_sch         # preparar un archivo
git add .                         # preparar todo
git commit -m "tipo(alcance): qué"  # tomar la foto
git push                          # subir a GitHub
git pull                          # bajar de GitHub
git log --oneline --graph         # ver el historial
```

Empezar un proyecto

```
git init                                          # proyecto nuevo
git remote add origin https://github.com/USUARIO/REPO.git
git push -u origin main                           # primer push

git clone https://github.com/USUARIO/REPO.git     # o copiar uno existente
```

Ramas

```
git branch                      # listar ramas (* = actual)
git switch -c feat/nombre       # crear una rama y cambiarse a ella
git switch main                 # cambiar de rama (¡cierra KiCad antes!)
git push -u origin feat/nombre  # subir la rama
git merge feat/nombre           # integrar la rama en la actual
git branch -d feat/nombre       # borrar una rama ya integrada
```

Deshacer

```
git restore archivo             # descartar cambios SIN commit de un archivo
git restore --staged archivo    # sacar del staging (no pierde cambios)
git rm -r --cached carpeta      # dejar de rastrear (ej. si olvidaste el .gitignore)
git revert <hash>               # nuevo commit que deshace otro (seguro)
```

Conflictos

```
git checkout --ours archivo     # quedarse con MI versión (rama actual)
git checkout --theirs archivo   # quedarse con la versión que estoy integrando
git add archivo
git commit -m "merge: resuelve conflicto en ..."
git merge --abort               # cancelar el merge y volver atrás
```

Mensajes de commit

```
feat(sch): agrega sensor de temperatura
fix(pcb): corrige footprint de U3
docs: actualiza README con pinout
chore: actualiza .gitignore
```

Tipos: `feat` `fix` `docs` `refactor` `test` `chore` · Alcances: `sch` `pcb` `lib` `bom` `mech` `fw`

Versiones de hardware

`MAJOR.MINOR.SUBREVISION`: **MAJOR** = forma, voltajes o interfaces · **MINOR** = esquemático o layout · **SUBREVISION** = BOM o valores

Reglas de oro en hardware

1. **Cierra KiCad** antes de `git switch`, `git pull` o `git merge` (aunque KiCad 10 muestre la rama, no recarga los archivos abiertos).
2. **Commits pequeños**: un cambio lógico por commit.

3. **Nunca** trabajes directo en `main`: rama → PR → revisión → merge.
4. Repártanse el trabajo **por hoja de esquemático o por zona del PCB** para evitar conflictos.
5. Las salidas de fabricación van al **Release**, no al commit.